Chapter 7

Classes and Object

C++ OOPs Concepts

The major purpose of C++ programming is to introduce the concept of object orientation to the C programming language.

Procedural programming is about writing procedures or functions that perform operations on the data, while object-oriented programming is about creating objects that contain both data and functions.

Advantage of OOPs over Procedure-oriented programming language

- OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.
- OOPs provide data hiding whereas in Procedure-oriented programming language a global data can be accessed from anywhere.
- OOPs provide ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time.

Object Oriented Programming is a paradigm that provides many concepts such as **inheritance, data binding, polymorphism etc.**

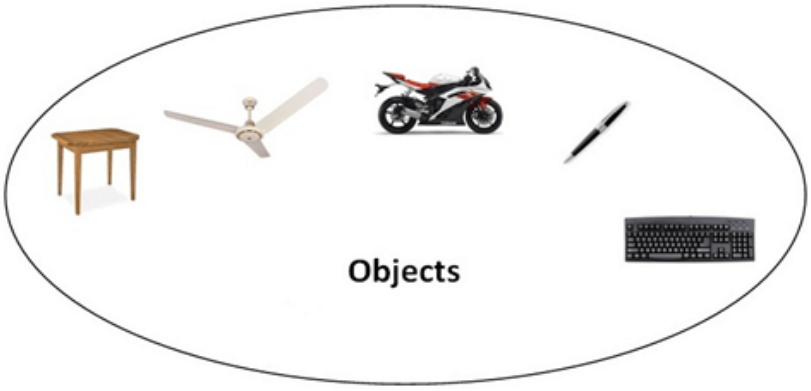
The programming paradigm where everything is represented as an object is known as truly object-oriented programming language. **Smalltalk** is considered as the first truly object-oriented programming language.

OOPs (Object Oriented Programming System)

Object means a real word entity such as pen, chair, table etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:

- Object
- Class

- Inheritance
- Polymorphism
- Abstraction
- Encapsulation



Object

Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

Class

Collection of objects is called class. It is a logical entity.

Inheritance

When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

When **one task is performed by different ways** i.e. known as polymorphism. For example: to convince the customer differently, to draw something e.g. shape or rectangle etc.

In C++, we use Function overloading and Function overriding to achieve polymorphism.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing.

In C++, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.

C++ Object and Class

Since C++ is an object-oriented language, program is designed using objects and classes in C++.

C++ Object

In C++, Object is a real world entity, for example, chair, car, pen, mobile, laptop etc.

In other words, object is an entity that has state and behavior. Here, state means data and behavior means functionality.

Object is a runtime entity, it is created at runtime.

Object is an instance of a class. All the members of the class can be accessed through object.

Let's see an example to create object of student class using s1 as the reference variable.

```
Student s1;  //creating an object of Student
```

In this example, Student is the type and s1 is the reference variable that refers to the instance of Student class.

C++ Class

In C++, class is a group of similar objects. It is a template from which objects are created. It can have fields, methods, constructors etc.

Let's see an example of C++ class that has three fields only.

```
class Student
 {
   public:
   int id;  //field or data member
   String course; //field or data member
   String name;//field or data member
 }
```

Let's see an example of class that has two fields: id and name. It creates instance of the class, initializes the object and prints the object value.

```
#include <iostream>
using namespace std;
class Student {
  public:
    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
};
int main() {
   Student s1; //creating an object of Student
   s1.id = 201;
   s1.name = "Sumit panwar";
   cout<<s1.id<<endl;
   cout<<s1.name<<endl;
   return 0;
}
```

Output:

```
201
Sumit Panwar
```

C++ Class Example: Initialize and Display data through method

Let's see another example of C++ class where we are initializing and displaying object through method.

```
#include <iostream>
using namespace std;
class Student {
  public:
    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
    void insert(int i, string n)
     {
        id = i;
        name = n;
     }
    void display()
     {
```

```
        cout<<id<<"  "<<name<<endl;
    }
};
int main(void) {
   Student s1; //creating an object of Student
   Student s2; //creating an object of Student
   s1.insert(201, "Sonoo");
   s2.insert(202, "Nakul");
   s1.display();
   s2.display();
   return 0;
}
```

Output:

```
201  Sonoo
202  Nakul
```

Let's see another example of C++ class where we are storing and displaying employee information using method.

```
#include <iostream>
using namespace std;
class Employee
{
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     float salary;
     void insert(int i, string n, float s)
      {
         id = i;
         name = n;
         salary = s;
      }
     void display()
      {
```

```
        cout<<id<<"  "<<name<<"  "<<salary<<endl;
    }
};
int main(void) {
   Employee e1; //creating an object of Employee
   Employee e2; //creating an object of Employee
   e1.insert(201, "Ankit",990000);
   e2.insert(202, "Nakul", 29000);
   e1.display();
   e2.display();
   return 0;
}
```

Output:

```
201  Ankit  990000
202  Nakul  29000
```

Class Methods

Methods are **functions** that belongs to the class.

There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

In the following example, we define a function inside the class, and we name it "myMethod".

Note: You access methods just like you access attributes; by creating an object of the class and using the dot syntax (.):

```
class MyClass {       // The class
  public:             // Access specifier
    void myMethod() {  // Method/function defined inside the class
      cout << "Hello World!";
    }
};

int main() {
```

```
  MyClass myObj;     // Create an object of MyClass
  myObj.myMethod();  // Call the method
  return 0;
}
```

To define a function outside the class definition, you have to declare it inside the class and then define it outside of the class. This is done by specifiying the name of the class, followed the scope resolution :: operator, followed by the name of the function:

Outside Example

```
class MyClass {       // The class
  public:             // Access specifier
    void myMethod();   // Method/function declaration
};

// Method/function definition outside the class
void MyClass::myMethod()

{
  cout << "Hello World!";
}

int main() {
  MyClass myObj;     // Create an object of MyClass
  myObj.myMethod();  // Call the method
  return 0;
}
```

You can also add parameters:

```
#include <iostream>
using namespace std;

class Car {
  public:
    int speed(int maxSpeed);
};

int Car::speed(int maxSpeed) {
  return maxSpeed;
}
```

```
int main() {
  Car myObj; // Create an object of Car
  cout << myObj.speed(200); // Call the method with an argument
  return 0;
}
```

Constructors

A constructor in C++ is a **special method** that is automatically called when an object of a class is created.

To create a constructor, use the same name as the class, followed by parentheses ():

```
class MyClass {     // The class
  public:           // Access specifier
    MyClass() {     // Constructor
      cout << "Hello World!";
    }
};

int main() {
  MyClass myObj;    // Create an object of MyClass (this will call the constructor)
  return 0;
}
```

Note: The constructor has the same name as the class, it is always public, and it does not have any return value.

Constructor Parameters

Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

The following class have brand, model and year attributes, and a constructor with different parameters. Inside the constructor we set the attributes equal to the constructor parameters (brand=x, etc). When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same:

```
class Car {       // The class
  private:        // Access specifier
    string brand;  // Attribute
```

```
  string model;  // Attribute
  int year;      // Attribute

public:
  Car(string x, string y, int z) { // Constructor with parameters
    brand = x;
    model = y;
    year = z;
  }
};

int main() {
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values
  cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
  return 0;
}
```

Just like functions, constructors can also be defined outside the class. First, declare the constructor inside the class, and then define it outside of the class by specifying the name of the class, followed by the scope resolution :: operator, followed by the name of the constructor (which is the same as the class):

```
class Car {       // The class
  public:         // Access specifier
    string brand;  // Attribute
    string model;  // Attribute
    int year;      // Attribute
    Car(string x, string y, int z); // Constructor declaration
};

// Constructor definition outside the class
Car::Car(string x, string y, int z) {
  brand = x;
  model = y;
  year = z;
}

int main() {
  // Create Car objects and call the constructor with different values
```

```
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values
  cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
  return 0;
}
```

Access Specifiers

By now, you are quite familiar with the public keyword that appears in all of our class examples:

```
class MyClass {  // The class
  public:       // Access specifier
    // class members goes here
};
```

The public keyword is an **access specifier.** Access specifiers define how the members (attributes and methods) of a class can be accessed. In the example above, the members are public - which means that they can be accessed and modified from outside the code.

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

- public - members are accessible from outside the class
- private - members cannot be accessed (or viewed) from outside the class
- protected - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn more about Inheritance later.

In the following example, we demonstrate the differences between public and private members:

```
class MyClass {
  public:    // Public access specifier
    int x;   // Public attribute
  private:   // Private access specifier
    int y;   // Private attribute
};

int main() {
  MyClass myObj;
  myObj.x = 25;  // Allowed (public)
```

```
  myObj.y = 50;  // Not allowed (private)
  return 0;
}
```

If you try to access a private member, an error occurs:

Note: It is possible to access private members of a class using a public method inside the same class.

By default, all members of a class are private if you don't specify an access specifier:

```
class MyClass {
  int x;   // Private attribute
  int y;   // Private attribute
};
```

Encapsulation

The meaning of **Encapsulation**, is to make sure that "sensitive" data is hidden from users. To achieve this, you must declare class variables/attributes as private (cannot be accessed from outside the class). If you want others to read or modify the value of a private member, you can provide public **get** and **set** methods.

Access Private Members

To access a private attribute, use public "get" and "set" methods:

```
#include <iostream>
using namespace std;

class Employee {
  private:
    // Private attribute
    int salary;

  public:
    // Setter
    void setSalary(int s) {
      salary = s;
    }
    // Getter
    int getSalary() {
      return salary;
    }
};
```

```
int main() {
  Employee myObj;
  myObj.setSalary(50000);
  cout << myObj.getSalary();
  return 0;
}
```

Explain Example

The salary attribute is private, which have restricted access.

The public setSalary() method takes a parameter (s) and assigns it to the salary attribute (salary = s).

The public getSalary() method returns the value of the private salary attribute.

Inside main(), we create an object of the Employee class. Now we can use the setSalary() method to set the value of the private attribute to 50000. Then we call the getSalary() method on the object to return the value.

Why Encapsulation?

- It is considered good practice to declare your class attributes as private (as often as you can). Encapsulation ensures better control of your data, because you (or others) can change one part of the code without affecting other parts
- Increased security of data

Understanding static

We have seen that a class basically contains two sections. One declares variables and other declares methods. These variables and method are called *instance variable* and *instance method*.

This is because every time the class is instantiated, a new copy of each of them is created. They are accessed using the objects (with dot operator).

There will be times when you will want to define a class member that will be used independently of any object of that class. That is, the member belongs to the class as whole rather than the objects created from the class. To create such a member, precede its declaration with the keyword **static**. You can declare both methods and variables to be **static**.

When we declare a member of a class as **static** it means no matter how many objects of the class are created, there is **only one copy of the static member**. This means **one single copy** of that data member is **shared** between **all objects of that class**.

Static data members can be initialized outside the class using the **scope resolution operator (::)** to identify which class it belongs to. Since Static data members are **class level variables**, we **do not require the object** to access these variables and they can be accessed simply by using the class name and **scope resolution(::) operator** with the variable name to access its value.

A static member has certain special characteristics. These are:

- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is initialized to zero when the first object of its class is created. No other initialization is permitted.
- It is visible only within the class, but its lifetime is the entire program
- It cannot be private.

Methods declared as **static** have some restrictions:

- They can only call other **static** methods.
- They must only access **static** data.
- They cannot refer to **this** in any way.

A very common use of static data members is to keep a count of total objects of a particular class Let's see example of static keyword in C++ which counts the objects.

```
#include <iostream>
using namespace std;
class Account {
  public:
     int accno; //data member (also instance variable)
     string name;
     static int count;
     Account(int accno, string name)
      {
          this->accno = accno;
         this->name = name;
         count++;
      }
     void display()
      {
         cout<<accno<<" "<<name<<endl;
```

```
    }
};
int Account::count=0;
int main(void) {
   Account a1 =Account(201, "Sanjay"); //creating an object of Account
   Account a2=Account(202, "Nikhil");
    Account a3=Account(203, "Renu");
   a1.display();
   a2.display();
   a3.display();
   cout<<"Total Objects are: "<<Account::count;
   return 0;
}
```

Let's see another example of static field in C++.

Static Function Members

By declaring a function member as static, you make it independent of any particular object of the class. A static member function can be called even if no objects of the class exist and the **static** functions are accessed using only the class name and the scope resolution operator **::**.

Let us try the following example to understand the concept of static function members:

```
#include <iostream>
using namespace std;
class Box {
  public:
    static int objectCount;

    // Constructor definition
    Box(double l = 2.0, double b = 2.0, double h = 2.0) {
      cout <<"Constructor called." << endl;
      length = l;
      breadth = b;
      height = h;

      // Increase every time object is created
      objectCount++;
    }
    double Volume() {
      return length * breadth * height;
```

```
    }
    static int getCount() {
      return objectCount;
    }

  private:
    double length;     // Length of a box
    double breadth;    // Breadth of a box
    double height;     // Height of a box
};

// Initialize static member of class Box
int Box::objectCount = 0;

int main(void) {
  // Print total number of objects before creating object.
  cout << "Inital Stage Count: " << Box::getCount() << endl;

  Box Box1(3.3, 1.2, 1.5);    // Declare box1
  Box Box2(8.5, 6.0, 2.0);    // Declare box2

  // Print total number of objects after creating object.
  cout << "Final Stage Count: " << Box::getCount() << endl;

  return 0;
}
```

Output:

```
Inital Stage Count: 0
Constructor called.
Constructor called.
Final Stage Count: 2
```

Class objects as static

Just like variables, objects also when declared as static have a scope till the lifetime of program.

Consider the below program where the object is non-static.

```
#include<iostream>
using namespace std;

class StaticDemo
{
```

```
    int i;
    public:
        StaticDemo()
        {
            i = 0;
            cout << "Inside Constructor\n";
        }
        ~StaticDemo()
        {
            cout << "Inside Destructor\n";
        }
};

int main()
{
    int x = 0;
    if (x==0)
    {
        StaticDemo obj;
    }
    cout << "End of main\n";
}
```

Output

Inside Constructor

Inside Destructor

End of main

In the above program the object is declared inside the if block as non-static. So, the scope of variable is inside the if block only. So when the object is created the constructor is invoked and soon as the control of if block gets over the destructor is invoked as the scope of object is inside the if block only where it is declared.

Let us now see the change in output if we declare the object as static.

```
#include<iostream>
using namespace std;

class StaticDemo
{
    int i = 0;

    public:
    StaticDemo()
    {
```

```
        i = 0;
        cout << "Inside Constructor\n";
    }

    ~StaticDemo()
    {
        cout << "Inside Destructor\n";
    }
};

int main()
{
    int x = 0;
    if (x==0)
    {
        static StaticDemo obj;
    }
    cout << "End of main\n";
}
```

Output:

Inside Constructor

End of main

Inside Destructor

You can clearly see the change in output. Now the destructor is invoked after the end of main. This happened because the scope of static object is throughout the life time of program.

Object as a Function Arguments

Like any other data type, an object may be used as a function argument. This can be done in two ways:

- A copy of the entire object is passed to the function.
- Only the address of the object is transferred to the function.

The first method is called *pass-by-value*. Since a copy of the object is passed to the function, any changes made to the object inside the function do not affect the object used to call the function. The second method is called *pass-by-reference*. When an address of the object is passed, the called function works directly on the actual object used in the call. This means that any changes made to the object inside the function will reflect in the actual object. The pass-by-reference method is more efficient since it requires to pass only the address of the object and not the entire object.

Following example perform the addition of time in the hour and minute format:

```
#include<iostream.h>
class time
{
     int hours,minutes;
     public:
     void gettime(int h,int m)
     {
        hours=h;
        minutes=m;
     }
     void puttime(void)
     {
         cout<<hours<< " hours and ";
         cout<<minutes<<" minutes "<<endl;
     }
     void sum(time, time);                    //declaration of function with object as argument
};

void time :: sum(time t1, time t2)
{
    minutes=t1.minutes+t2.minutes;
    hours=minutes/60;
    minutes=minutes%60;
    hours=hours+t1.hours+t2.hours;
}
int main()
{
    time T1,T2, T3;
    T1.gettime(2, 45);
    T2.gettime(3, 30);
    T3.sum(T1,T2);  //T3=Tj1+T2;
    cout<<" T1= ";
    T1.puttime();
    cout<<"T2= ";
    T2.puttime();
    cout<<"T3= ";
    T3.puttime();
    return 0;
}
```

Output

T1= 2 hourss and 45 minutes
T2= 3 hourss and 30 minutes
T3= 6 hourss and 15 minutes

Since the member function **sum()** is invoked by the object T3, with the object T1 and T2 as arguments, it can directly access the hours and minutes variables of T3. But, the members of T1 and T2 can be accessed only by using the dot operator (like T1.hours and T1.minutes). Therefore, inside the function sum(), the variable hours and minutes refer to T3, T1.hours and T1.minutes refer to T1, and T2.hours and T2.minutes refer to T2.

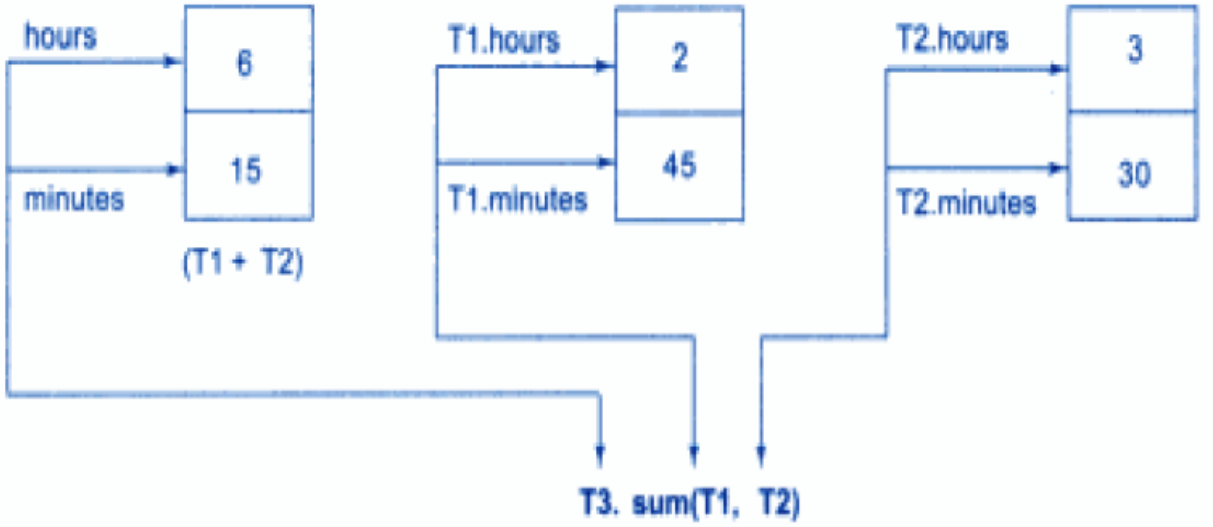


Figure: Accessing member of the object within a function

Dynamic Memory in C++

Suppose you want to put a toy in a box, but you only have an approximate idea of its size. For that, you would require a box whose size is equal to the approximate size of the toy.

We face a similar situation in C++ also when we want to input a sentence as an array of characters but are not sure about the number of characters in the array.

Now, while declaring the character array, if we specify its size smaller than the size of the input string, then we will get an error because the space in the memory allocated to the array is lesser than the size of the input string. This is the same case as trying to fit a big toy in a smaller box. If we specify its size much larger than the size of the input string, then the array will be allocated a space in the memory which is much larger than the size of the input string, thus unnecessarily consuming more memory even when it is not required. This is like putting a small toy in a large box.

In the above case, we don't have the idea about the size of the array until the compile time (when computer compiles the code and the string is input by the user). In such cases, we use

the **new** operator.

But before going to the new operator, let's have a look at the two parts in which our memory is divided. These parts are as follows:

- **stack -** Memory from the stack is used by all the members which are declared inside functions. Note that main is also a function.
- **heap -** This memory is unused and can be used to dynamically allocate the memory at runtime.

What is Dynamic Memory Allocation?

Programmers can dynamically allocate storage space while the program is running, but programmers cannot create new variable names "on the fly", and for this reason, dynamic allocation requires two criteria:

- Creating the dynamic space in memory
- Storing its address in a pointer (so that space can be accessed)

Memory de-allocation is also a part of this concept where the "clean-up" of space is done for variables or other data storage. It is the job of the programmer to de-allocate dynamically created space. For de-allocating dynamic memory, we use the **delete** operator. In other words, dynamic memory allocation refers to performing memory management for dynamic memory allocation manually.

new

The **new** operator is used to **allocate memory at runtime**. The memory is allocated in bytes.

Let's first see how to allocate a variable dynamically.

```
int *ptr = new int;
```

By writing **new int**, we allocated the space in memory required by an integer. Then we assigned the address of that memory to an integer pointer **ptr**.

We assign value to that memory as follows:

```
*ptr = 4;
```

Thus, we allocated that much space in memory that would be required by an **int** and then assigned the address of that memory to a pointer **ptr** and assigned the memory a value **4**.

When we dynamically allocate some memory to a variable, we actually use the heap memory.

We can initialize a variable while dynamical allocation in the following two ways.

```
int *ptr = new int (4);

int *ptr = new int {4};
```

Let's see an example.

```
#include <iostream>
int main()
{
        int *ptr = new int;
        *ptr = 4;
        std::cout << *ptr << std::endl;
        return 0;
}
```

Output:
4

We just combined all the above steps in this example.

So, we have just seen how to dynamically allocate a variable. Now let's allocate arrays dynamically.

Dynamically Allocating Arrays

The main use of the concept of dynamic memory allocation is for allocating arrays when we have to declare an array by specifying its size but are not sure about the size.

Consider a situation when we want the user to enter the name but are not sure about the number of characters in the name that the user will enter. In that case, we will declare an array of characters for the name with some array size such that the array size should be sufficient enough to hold any name entered. Suppose we declared the array with the array size 30 as follows.

```
char name[30];
```

And if the user enters the name having only 12 characters, then the rest of the memory space which was allocated to the array at the time of its declaration would become waste, thus unnecessary consuming the memory.

In this case, we will be using the **new** operator to dynamically allocate the memory at runtime. We use the new operator as follows.

```
char *arr = new char[length];
```

Let's see an example to understand its use.

```
#include <iostream>

using namespace std;

int main()
{
  int length, sum = 0;
  cout << "Enter the number of students in the group" << endl;
  cin >> length;
  int *marks = new int[length];
  cout << "Enter the marks of the students" << endl;
 for( int i = 0; i < length; i++ )// entering marks of students
            {
                        cin >> *(marks+i);
            }
            for( int i = 0; i < length; i++ )// calculating sum
            {
                        sum += *(marks+i);
            }
            cout << "sum is " << sum << endl;
            return 0;
}
```

Output

```
Enter the number of students in the group
4
Enter the marks of the students
65
47
74
45
sum is 231
```

In this example, we are calculating the sum of the marks of all the students of a group. Since different groups have a different number of students, therefore we are asking the number of students (i.e. the size of the array) every time we are running the program.

In this example, the user entered the size as 4.

int *marks = new int[length]; - We declared an array of integer and allocated it some space in

memory dynamically equal to the size which would be occupied by **length** number of **integers**. Thus it is allocated a space equal to **'length * (size of 1 integer)'** and assigned the address of the assigned memory to the pointer **marks**. The rest of the steps must be clear to you.

We call this array **dynamic** because it is being assigned memory when the program runs. We made this possible by using the **new** operator. This dynamic array is being allocated memory from heap unlike other fixed arrays which are provided memory from stack. We can give any size to these dynamic arrays and there is no limitation to it.

Now what if the variable to which we dynamically allocated some memory is not required anymore?

In that case, we use the **delete** operator.

delete

Suppose we allocated some memory to a variable dynamically and then we realize that the variable is not needed anymore in the program. In that case, we need to free the memory which we had assigned to that variable. For that, we use the **delete** operator.

It is advised to free the dynamically allocated memory after the program finishes so that it becomes available for future use.

To delete the memory assigned to a variable, we simply need to write the following code.

delete ptr;

Here ptr is the pointer to the dynamically allocated variable.

There is nothing much to understand in this. The delete operator simply returns the memory allocated back to the operating system so that it can be used again. Let's look at an example.

After printing the value 4, we deleted the pointer i.e. deleted the address of the allocated memory thus freeing it.

Once deleted, a pointer will point to deallocated memory and will be called a **dangling pointer**. If we further try to delete a dangling pointer, we will get some undefined behavior.

Deleting Array

To delete an array which has been allocated in this way, we write the following code.

delete[] ptr;

Here ptr is a pointer to an array which has been dynamically allocated.

```
#include <iostream>

using namespace std;
int main()
{
  int length, sum = 0;
  cout << "Enter the number of students in the group" << endl;
  cin >> length;
  int *marks = new int[length];
  cout << "Enter the marks of the students" << endl;
  for( int i = 0; i < length; i++ )// entering marks of students
            {
              cin >> *(marks+i);
            }
            for( int i = 0; i<length; i++ ) // calculating sum
            {
                        sum += *(marks+i);
            }
            cout << "sum is " << sum << endl;
            delete[] marks;
            return 0;
}
```

Output

```
Enter the number of students in the group
4
Enter the marks of the students
65
47
74
45
sum is 231
```

We just wrote **delete[ ] marks;** at the end of the program to release the memory which was dynamically allocated using new.

Dynamic Memory Allocation for Objects

We can also dynamically allocate objects.

As we know that **Constructor** is a member function of a class which is called whenever a new object is created of that class. It is used to initialize that object. **Destructor** is also a class member function which is called whenever the object goes out of scope.

Destructor is used to release the memory assigned to the object. It is called in these conditions.

- When a local object goes out of scope
- For a global object, operator is applied to a pointer to the object of the class

We again use pointers while dynamically allocating memory to objects.

Let's see an example of array of objects.

```
#include <iostream>
using namespace std;
class stud
{
public:
   stud()
   {
      cout << "Constructor Used" << endl;
   }
   ~stud()
   {
      cout << "Destructor Used" << endl;
   }
};

int main()
{
   stud* S = new stud[6];
   delete[] S;
}
```

Another Example

```
#include <iostream>
using namespace std;

class Box {
  public:
     Box() {
        cout << "Constructor called!" <<endl;
     }
     ~Box() {
        cout << "Destructor called!" <<endl;
     }
};
int main() {
  Box* myBoxArray = new Box[4];
  delete [] myBoxArray; // Delete array

  return 0;
```

}

If you were to allocate an array of four Box objects, the Simple constructor would be called four times and similarly while deleting these objects, destructor will also be called same number of times.

If we compile and run above code, this would produce the following result −

```
Constructor called!
Constructor called!
Constructor called!
Constructor called!
Destructor called!
Destructor called!
Destructor called!
Destructor called!
```

Constructor overloading in C++

Constructor is a member function of a class that is used to initialize the objects of the class. Constructors do not have any return type and are automatically called when the object is created.

Characteristics of constructors

- The name of the constructor is the same as the class name
- No return type is there for constructors
- Called automatically when the object is created
- Always placed in the public scope of the class
- If a constructor is not created, the default constructor is created automatically and initializes the data member as zero
- Declaration of constructor name is case-sensitive
- A constructor is not implicitly inherited

Let first understand the types of constructors

Types of constructors

There are three types of constructors -

Default constructor - A default constructor is one that does not have function parameters. It is used to initialize data members with a value. The default constructor is called when the object is created.

```
#include <iostream>
using namespace std;
class construct // create a class construct
{
public:
   int a, b; // initialise two data members
   construct() // this is how default constructor is created
   {
      // We can also assign both values to zero
      a = 10; // intialise a with some value
      b = 20; // initialise b with some value
   }
};
int main()
{
   construct c; // creating an object of construct calls defualt constructor
   cout<< "a:" <<c.a<<endl
      << "b:" <<c.b; // print a and b
   return 1;
}
```

Output

a:10
b:20

Parameterized constructor - A non-parameterized constructor does have the constructor arguments and the value passed in the argument is initialized to its data members. Parameterized constructors are used in constructor overloading.

```
#include <iostream>
using namespace std;
class Point // create point class
  {
private:
int x, y; // the two data members of class Point
public:
Point(int x1, int y1) // create paramterised Constructor and initialise data member
  {
    x = x1; // x1 is now intialised to x
    y = y1; // y1 is now intialised to y
  }
intgetX()
```

```
   {
return x; // to get the value of x
   }
intgetY()
   {
return y; // to get the value of y
   }
};
int main()
{
   Point p1(10, 15); // created object for paramterised constructor

cout<< "p1.x = " << p1.getX() << ", p1.y = " << p1.getY(); // print x and y

return 0;
}
```

Output

p1.x = 10, p1.y = 15

p1.x = 10, p1.y = 15

Explanation

Create a class point with two data members x and y. A parameterized constructor is created with the points x1 and y1 as parameters and the value of x and y are assigned using x1 and y1. In the main function, we create a parameterized constructor with the values (10, 15). Using the getter functions, we get the values of the data members.

Copy constructor - Copy constructor initializes an object using another object of the same class.

Syntax

class_name(const classname&old_object).

```
#include <iostream>
using namespace std;
class Point // create point class
   {
private:
int x, y; // data members of the class
public:
Point(int x1, int y1)
   {
```

```
        x = x1;
        y = y1;
    } // parameterised constructor

    // Copy constructor
Point(const Point& p1) // initialisation according to syntax
    {
        x = p1.x;
        y = p1.y;
    }
intgetX() { return x; } // return value of x
intgetY() { return y; } // return valur of y
};
int main()
{
    Point p1(10, 15); // call parameterised constructor
    Point p2 = p1; // call Copy constructor
    //  use getter and setter to print x and y
cout<< "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
cout<< "\np2.x = " << p2.getX() << ", p2.y = " << p2.getY();
return 0;
}
p1.x = 10, p1.y = 15
p2.x = 10, p2.y = 15
```

Constructor Overloading

As there is a concept of function overloading, similarly constructor overloading is applied. When we overload a constructor more than a purpose it is called constructor overloading.

The declaration is the same as the class name but as they are constructors, there is no return type.

The criteria to overload a constructor is to differ the number of arguments or the type of arguments.

```
// C++ program to demonstrate constructor overloading
#include <iostream>
using namespace std;
class Person { // create person class
private:
int age; // data member
```

```
public:
   // 1. Constructor with no arguments
Person()
   {
age = 20; // when object is created the age will be 20
   }
   // 2. Constructor with an argument
Person(int a)
{ // when parameterised Constructor is called with a value the
      // age passed will be initialised
age = a;
   }
intgetAge()
{ // getter to return the age
return age;
   }
};
int main()
{
   Person person1, person2(45); // called the object of person class in differnt way

cout<< "Person1 Age = " << person1.getAge() <<endl;
cout<< "Person2 Age = " << person2.getAge() <<endl;
return 0;
}
```

Output

```
Person1 Age = 20
Person2 Age = 45
```

In the above program, we have created a class **Person** with one data member(age). There are two constructors in the class that are overloaded. We have overloaded the second constructor by providing one argument and making it parameterized.

So, in the main function when the object person1 is created, it calls the non-parameterized constructor and when person2 is created, it calls the parameterized constructor and performs the required operation of initializing the age. Hence, when object person1 age is printed it gives 20 that was set by default and person2 age gives 45 as it was passed by the object to the parameterized constructor.

Another Example of Constructor Overloading

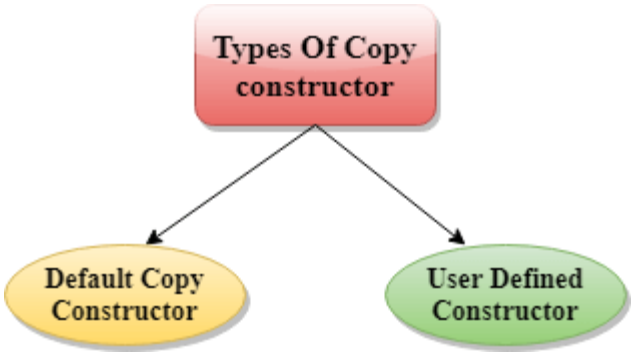
```
/*.....A program to feature the concept of constructor overloading..... */
#include <iostream>
using namespace std;
class ABC
{
    private:
      int x,y;
    public:
      ABC ()       //constructor 1 with no arguments
      {
          x = y = 0;
      }
      ABC(int a)    //constructor 2 with one argument
      {
          x = y = a;
      }
      ABC(int a,int b)    //constructor 3 with two argument
      {
           x = a;
           y = b;
      }
      void display()
      {
           cout << "x = " << x << " and " << "y = " << y << endl;
      }
};

int main()
{
    ABC cc1; //constructor 1
    ABC cc2(10); //constructor 2
    ABC cc3(10,20); //constructor 3
    cc1.display();
    cc2.display();
    cc3.display();
    return 0;
 }  //end of program
```

Insight of Copy Constructor

A Copy constructor is an **overloaded** constructor used to declare and initialize an object from another object.

Copy Constructor is of two types:

- **Default Copy constructor:** The compiler defines the default copy constructor. If the user defines no copy constructor, compiler supplies its constructor.
- **User Defined constructor:** The programmer defines the user-defined constructor.



Syntax Of User-defined Copy Constructor:

Class_name(**const** class_name &old_object);

```
class A
{
   A(A &x) //  copy constructor.
  {
     // copyconstructor.
  }
}
```

In the above case, **copy constructor can be called in the following ways:**

- **A a2(a1);**
- **A a2 = a1;**

} **a1 initialises the a2 object.**

Let's see a simple example of the copy constructor.

```
#include <iostream>
using namespace std;
```

```
class A
{
  public:
   int x;
   A(int a)              // parameterized constructor.
   {
     x=a;
   }
   A(A &i)               // copy constructor
   {
      x = i.x;
   }
};
int main()
{
  A a1(20);             // Calling the parameterized constructor.
 A a2 = a1;              //  Calling the copy constructor.
 cout<<a2.x;
  return 0;
}
```

Output:

20

When Copy Constructor is called

Copy Constructor is called in the following scenarios:

- When we initialize the object with another existing object of the same class type. For example, Student s1 = s2, where Student is the class.
- When the object of the same class type is passed by value as an argument.
- When the function returns the object of the same class type by value.

Two types of copies are produced by the constructor:

- Shallow copy
- Deep copy

Shallow Copy

- The default copy constructor can only produce the shallow copy.

- A Shallow copy is defined as the process of creating the copy of an object by copying data of all the member variables as it is.

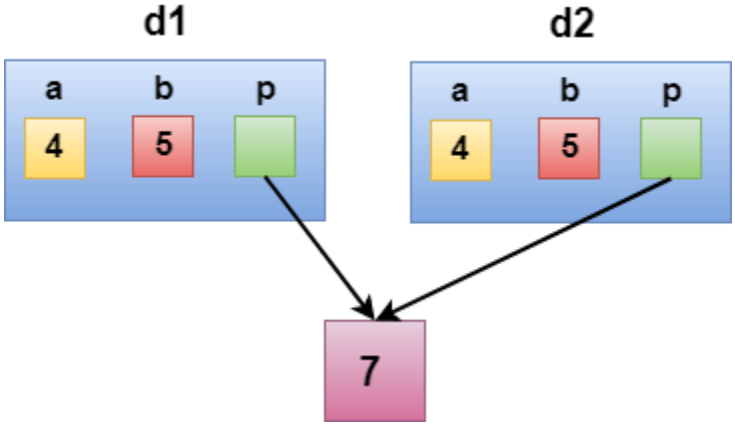
Let's understand this through a simple example:

```
#include <iostream>
 using namespace std;
class Demo
{
   int a;
   int b;
   int *p;
   public:
   Demo()
   {
      p=new int;
   }
   void setdata(int x,int y,int z)
   {
      a=x;
      b=y;
      *p=z;
   }
   void showdata()
   {
      std::cout << "value of a is : " <<a<< std::endl;
      std::cout << "value of b is : " <<b<< std::endl;
      std::cout << "value of *p is : " <<*p<< std::endl;
   }
};
int main()
{
  Demo d1;
  d1.setdata(4,5,7);
  Demo d2 = d1;
  d2.showdata();
    return 0;
}
```

Output:

```
value of a is : 4
```

```
value of b is : 5
value of *p is : 7
```



In the above case, a programmer has not defined any constructor, therefore, the statement **Demo d2 = d1;** calls the default constructor defined by the compiler. The default constructor creates the exact copy or shallow copy of the existing object. Thus, the pointer p of both the objects point to the same memory location. Therefore, when the memory of a field is freed, the memory of another field is also automatically freed as both the fields point to the same memory location. This problem is solved by the **user-defined constructor** that creates the **Deep copy**.

Deep copy

Deep copy dynamically allocates the memory for the copy and then copies the actual value, both the source and copy have distinct memory locations. In this way, both the source and copy are distinct and will not share the same memory location. Deep copy requires us to write the user-defined constructor.

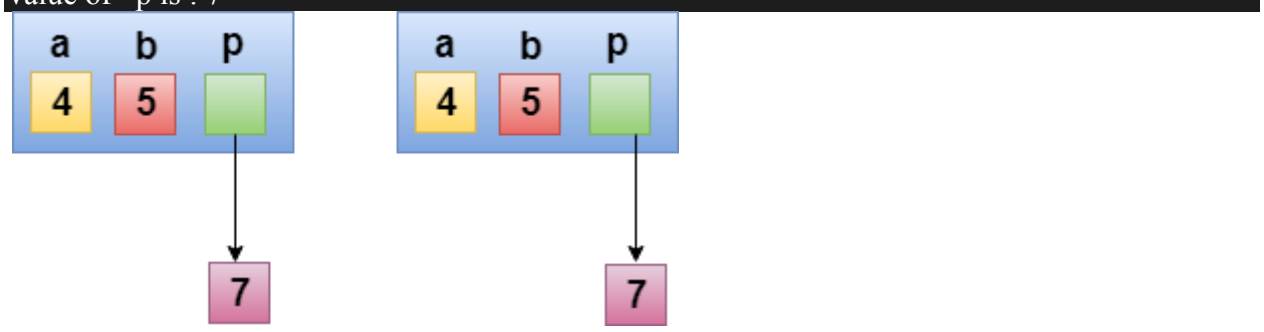
Let's understand this through a simple example.

```
#include <iostream>
using namespace std;
class Demo
{
   public:
   int a;
   int b;
   int *p;

   Demo()
   {
      p=new int;
   }
   Demo(Demo &d)
```

```
    {
       a = d.a;
       b = d.b;
       p = new int;
       *p = *(d.p);
    }
    void setdata(int x,int y,int z)
    {
       a=x;
       b=y;
       *p=z;
    }
    void showdata()
    {
       std::cout << "value of a is : " <<a<< std::endl;
       std::cout << "value of b is : " <<b<< std::endl;
       std::cout << "value of *p is : " <<*p<< std::endl;
    }
};
int main()
{
  Demo d1;
  d1.setdata(4,5,7);
  Demo d2 = d1;
  d2.showdata();
  return 0;
}
```

Output:

```
value of a is : 4
value of b is : 5
value of *p is : 7
```



In the above case, a programmer has defined its own constructor, therefore the statement **Demo d2 = d1;** calls the copy constructor defined by the user. It creates the exact copy of the value types data and the object pointed by the pointer p. Deep copy does not create the copy of a reference type variable.